Makefile (0,5 puntos)

Crea un fichero **Makefile** que permita generar todos los programas del enunciado a la vez y cada uno por separado. Añade una regla (clean) para borrar todos los binarios y/o ficheros objeto, y dejar solo los ficheros fuente. Los programas tienen que generarse si, y solo si, ha habido cambios en los ficheros fuente.

Control de errores (0,5 puntos)

Para todos los programas que se piden a continuación hay que comprobar los errores de TODAS las llamadas a sistema (excepto el write por pantalla), controlar los argumentos de entrada y definir la función Usage() si es necesario.

Ejercicio 1 (1,5 puntos)

Haz un programa llamado **Npidstat.c** para mostrar por pantalla las estadísticas sobre el uso de la CPU de un proceso. El programa tiene que recibir como primer argumento *COUNT*, un número que hay que comprobar que tenga un valor entre 1 y 60 (ambos incluidos), y después el identificador de proceso (PID) que se quiera monitorizar. No hay que comprobar que este PID sea un número válido, ni que el proceso exista. Para mostrar las estadísticas, el programa creará un único proceso hijo que ha de mutar su código a la línea de comandos '`pidstat -h -u -p <PID> 1 <COUNT>`'. Este comando muestra el uso de la CPU por pantalla tantas veces como indique *COUNT* en intervalos de un segundo. Puedes encontrar el programa `pidstat` adjunto al examen. Antes de acabar, el programa tiene que esperar al proceso hijo con una espera activa (sin capturar ningún signal para ello) y sin recoger su código de finalización.

Ejemplos de salida:

```
$ ./Npidstat 2 1
# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
18:51:08        0         1    0,00    0,00    0,00    0,00    0,00     1  systemd
18:51:09        0         1    0,00    0,00    0,00    0,00    0,00     1  systemd

$ ./Npidstat 61 1
Usage: Npidstat <COUNT> <PID>, 1 <= <COUNT> <= 60

$ ./Npidstat 3
Usage: Npidstat <COUNT> <PID>, 1 <= <COUNT> <= 60
```

ATENCIÓN! Este ejercicio se evaluará en función de la ejecución correcta del programa: nota del 100% si funciona según sus especificaciones; del 75% si hay algún error no relacionado con la asignatura; o del 0% si hay algún error relacionado con aspectos de sistemas operativos.

Ejercicio 2 (5,25 puntos)

Haz un programa llamado **scheduler.c** para planificar el uso de la CPU de un conjunto de procesos. Este programa tiene que recibir un argumento, un número con valor entre 1 y 60 (ambos incluidos) que corresponderá con la duración del periodo (en segundos) durante el cual se tienen que planificar los procesos. Después de comprobar los parámetros, el programa *scheduler* debe comportarse tal como se describe a continuación:

Proceso inicial:

- Tiene que crear de manera secuencial 3 procesos a planificar.
- Para cada proceso creado, tiene que crear un proceso adicional y mutarlo al programa *Npidstat* del ejercicio anterior, pasándole como argumentos el parámetro *T* y el PID del proceso n-ésimo creado del conjunto de procesos hijo secuenciales. Si no te ha funcionado el programa *Npidstat*, puedes usar el programa *Npidstat_ok* que se adjunta al examen.

- Tiene que esperar a que el proceso hijo a planificar esté creado y parado (pista: consulta el comportamiento del flag `WUNTRACED` de la llamada `waitpid`).
- Mientras no se agote el periodo de tiempo definido por el argumento *T*, tiene que reanudar la ejecución del proceso n-ésimo (mediante el envío del signal `SIGCONT`), esperar un segundo utilizando una alarma (sin consumir CPU) y volver a parar el proceso.
- Una vez agotado el periodo de tiempo definido por el argumento *T*, tiene que esperar un segundo adicional (también usando una alarma y sin consumir CPU) y después tiene que enviar al proceso n-ésimo a planificar el signal `SIGUSR2` y reanudar su ejecución con `SIGCONT`.
- Tiene que esperar de manera pasiva sus procesos hijo (sin capturar el signal SIGCHLD), sin recoger su código de finalización. Es decir, el proceso n-ésimo y el adicional.
- Las reprogramaciones de signals que sean necesarias se tienen que hacer con el signal en cuestión bloqueado en la máscara del proceso.

Cada proceso hijo a planificar:

- Después de hacer las inicializaciones y reprogramaciones de signals que corresponda, tiene que parar su ejecución mediante el envío a si mismo de un signal `SIGSTOP`. Como en el caso anterior, las reprogramaciones de signals se tienen que hacer con el signal en cuestión bloqueado en la máscara del proceso.
- Tiene que esperar la recepción del signal `SIGUSR2` mediante una espera activa.
- Cuando reciba el signal, tiene que acabar su ejecución retornando 0.

Cada proceso hijo adicional:

- Una vez creado, tiene que asegurarse que ha desbloqueado todos los signals.

Ejemplos de salida:

```
$ ./scheduler 3
# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
07:03:58     1000      3166  100.00    0.00    0.00    0.00  100.00    14  scheduler
07:03:59     1000      3166  100.00    0.00    0.00    0.00  100.00    14  scheduler
07:04:00     1000      3166   96.00    0.00    0.00    0.00   96.00    14  scheduler

# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
07:04:02     1000      3169  101.00    0.00    0.00    0.00  101.00     1  scheduler
07:04:03     1000      3169   99.01    0.00    0.00    0.00   99.01     1  scheduler
07:04:04     1000      3169   99.00    0.00    0.00    0.00   99.00     1  scheduler

# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
07:04:06     1000      3172  100.00    0.00    0.00    0.00  100.00    14  scheduler
07:04:07     1000      3172  100.00    0.00    0.00    0.00  100.00    14  scheduler
07:04:08     1000      3172   99.00    0.00    0.00    0.00   99.00    14  scheduler

$ ./scheduler
Usage: scheduler <T>, 1<= T <= 60

$ ./scheduler 61
Usage: scheduler <T>, 1<= T <= 60
```

Ejercicio 3 (2,25 puntos)

Copia el programa anterior, llámalo **scheduler_plus.c**, y modifícalo tal como se describe a continuación:

Proceso inicial:

- Para los 3 procesos planificados (sólo para los planificados, no para el adicional), tiene que recoger su estado de finalización (proceso acabado por exit o por signal) y el valor de finalización (código del exit o número de signal) y escribir esta información por pantalla, tal como se muestra en la parte final del ejemplo de salida siguiente.

Cada proceso hijo a planificar:

- Después de recibir el signal `SIGUSR2`, tiene que calcular cuánto tiempo total de CPU ha consumido el proceso mientras estaba en espera activa. Para hacerlo, debe utilizar la función `get_cpu_usage`, que se adjunta a este examen como un fichero objeto y que retorna (en formato `time_t` que equivale a `long int`) la cantidad total de microsegundos de CPU consumidos por el proceso que la invoca. Si la cantidad de microsegundos en espera activa es superior a 2000000 (es decir, 2 segundos), el proceso tiene que acabar su ejecución enviándose a si mismo un signal SIGKILL. De lo contrario, tiene que invocar `exit()` pasando como código de finalización la cantidad de microsegundos de CPU consumidos después de redondear a la cantidad en segundos más próxima.

Ejemplo de salida:

```
$ ./scheduler_plus 2
# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
10:01:19     1000      3833   98.04    0.00    0.00    0.00   98.04    14  scheduler_plus
10:01:20     1000      3833   96.08    0.00    0.00    0.00   96.08    14  scheduler_plus
El proceso 3833 ha acabado por exit con codigo 2

# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
10:01:22     1000      3836   99.01    0.00    0.00    0.00   99.01    14  scheduler_plus
10:01:22     1000      3836   95.15    0.00    0.00    0.00   95.15    14  scheduler_plus
El proceso 3836 ha acabado por exit con codigo 2

# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
10:01:24     1000      3839   98.02    0.00    0.00    0.00   98.02     1  scheduler_plus
10:01:25     1000      3839   98.02    0.00    0.00    0.00   98.02     1  scheduler_plus
El proceso 3839 ha acabado por exit con codigo 2


$ ./scheduler_plus 3
# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
10:07:00     1000      3897   98.00    0.00    0.00    0.00   98.00    14  scheduler_plus
10:07:01     1000      3897   98.00    0.00    0.00    0.00   98.00    14  scheduler_plus
10:07:02     1000      3897   98.00    0.00    0.00    0.00   98.00    14  scheduler_plus
El proceso 3897 ha acabado por el signal 9

# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
10:07:05     1000      3900   99.00    0.00    0.00    0.00   99.00    14  scheduler_plus
10:07:06     1000      3900   97.06    0.00    0.00    0.00   97.06    14  scheduler_plus
10:07:07     1000      3900   97.06    0.00    0.00    0.00   97.06    14  scheduler_plus
El proceso 3900 ha acabado por el signal 9

# Time        UID       PID    %usr %system  %guest   %wait    %CPU   CPU  Command
10:07:09     1000      3903   97.00    0.00    0.00    0.00   97.00    14  scheduler_plus
10:07:10     1000      3903   98.02    0.00    0.00    0.00   98.02    14  scheduler_plus
10:07:11     1000      3903   98.04    0.00    0.00    0.00   98.04    14  scheduler_plus
El proceso 3903 ha acabado por el signal 9
```

Qué se tiene que hacer

- El Makefile
- Los códigos de los programas en C
- La función Usage() para cada programa que sea necesario

Qué se valora

- Que se sigan las especificaciones del enunciado
- Que el uso de las llamadas al sistema sea correcto
- Que se comprueben los errores de todas las llamadas al sistema
- Que el código sea claro y correctamente indentado
- Que el Makefile tenga bien definidas las dependencias y objetivos
- Que la función Usage() muestre por pantalla como se tiene que invocar correctamente el programa en caso de que los argumentos recibidos no sean los adecuados

Qué hay que entregar

Un único fichero tar.gz con el código de todos los programas y el Makefile:

```
tar zcvf clab.tar.gz Makefile *.c
```